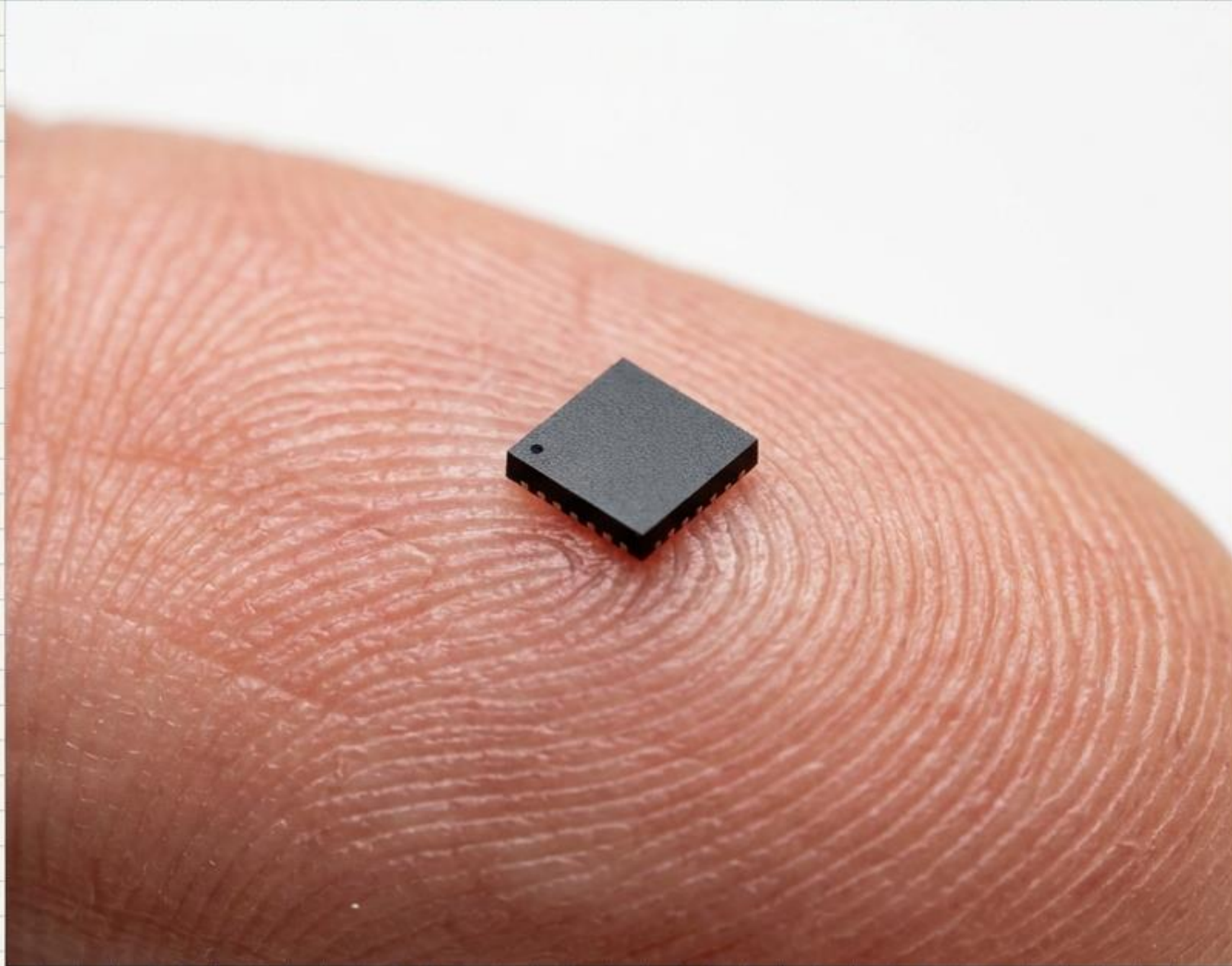




The Core Fundamentals

Brains, Senses, and Muscles in CircuitPython



The Tiny Computer

A microcontroller is a tiny, self-contained computer on a single chip. It does not need an external hard drive, RAM, or motherboard —everything required to think and act is baked into a piece of silicon the size of a fingernail.

The Generalist vs. The Specialist



The Big Computer

Designed to do a million things at once:
browse YouTube, play games, and write
essays.

Trait: Flexible, but complex.



The Tiny Brain

Designed to do one specific job perfectly,
over and over again, forever.

Trait: Hidden everywhere (timing the
spin cycle, spinning the microwave
plate, activating car brakes).

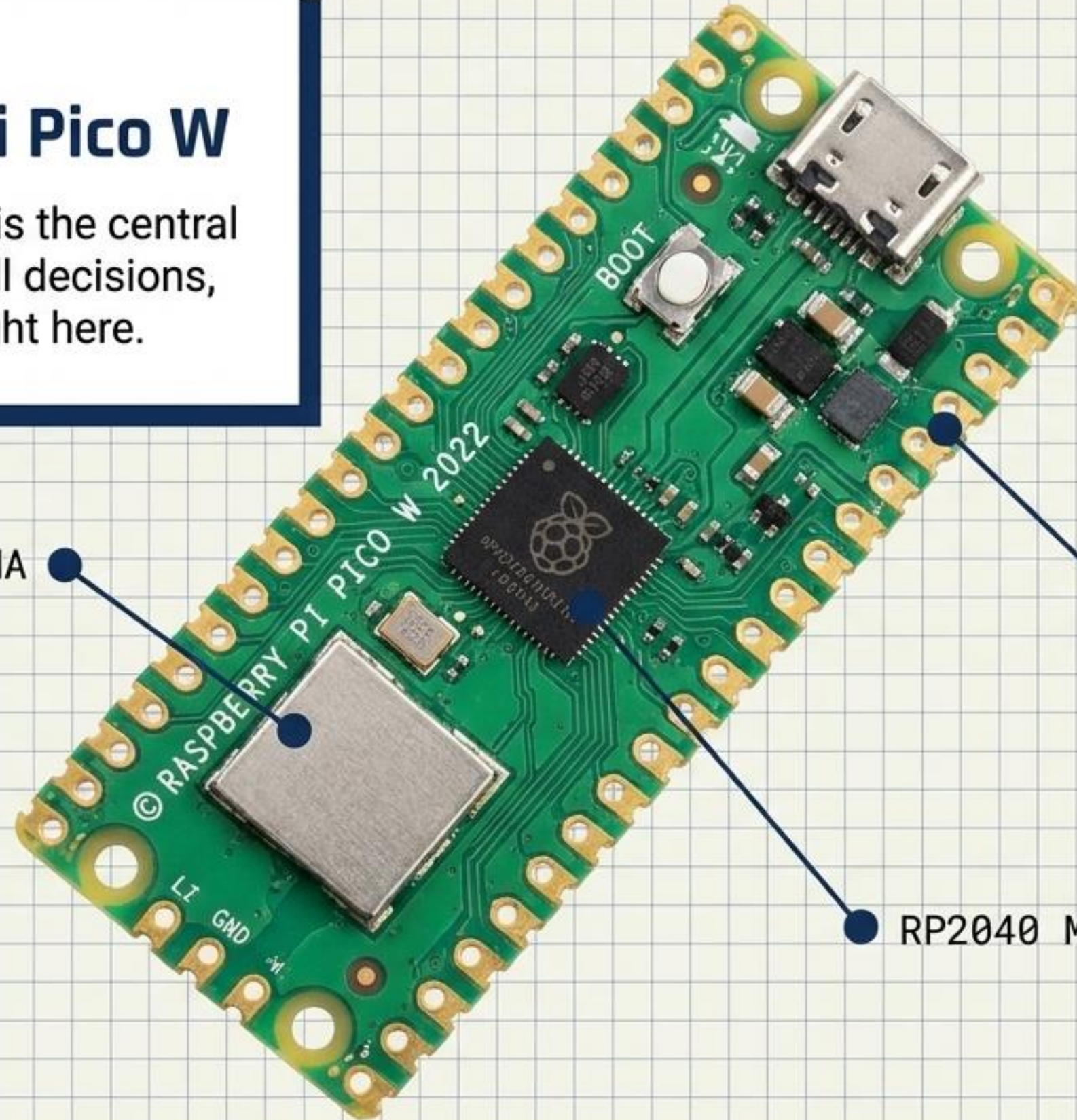
Meet the Brain: The Raspberry Pi Pico W

In this course, the Pico W is the central brain of our IoT system. All decisions, code, and logic happen right here.

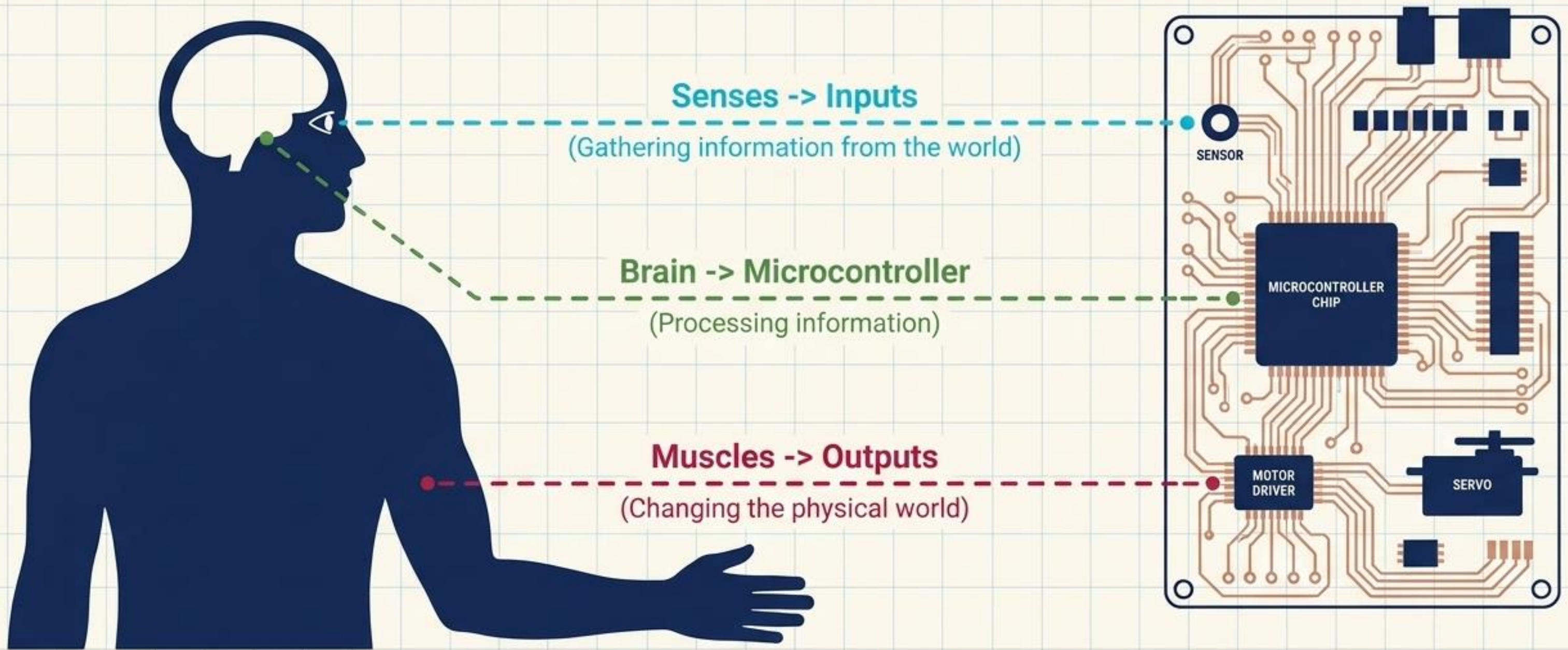
WI-FI/BLE ANTENNA

GPIO PINS

RP2040 MICROCONTROLLER CHIP

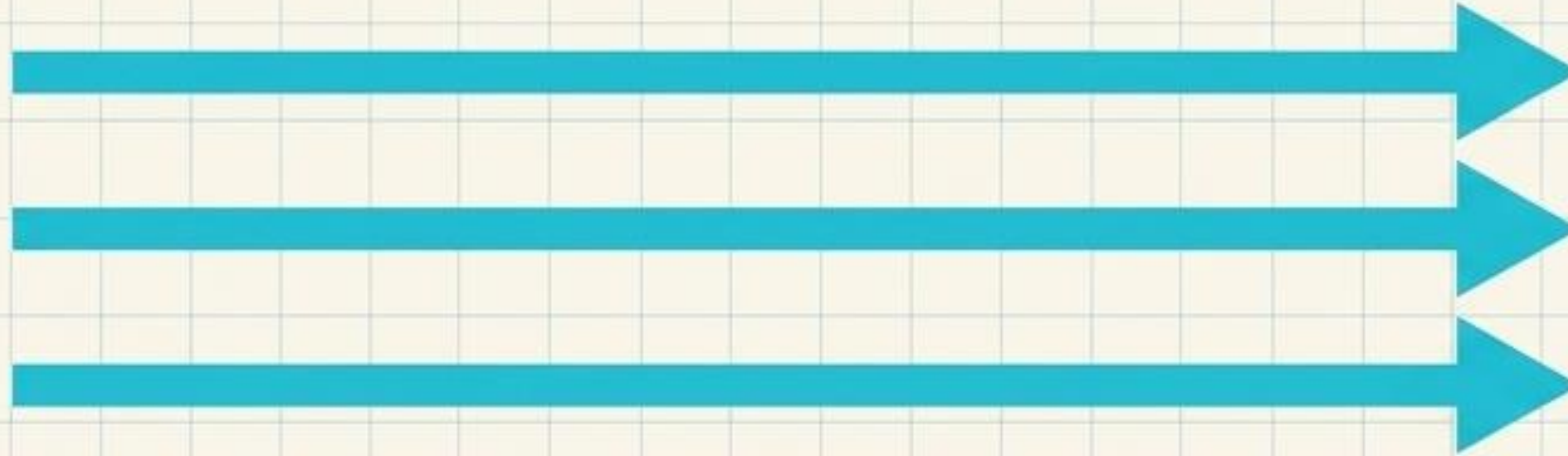


Hardware as a Human Body



Inputs (The Senses)

Inputs are how the microcontroller gathers information about the world around it. Just like humans use eyes to see and skin to feel, our Pico W uses Sensors.



**INPUTS = ELECTRICITY FLOWING
INTO THE PICO W**



The Sensory Hardware



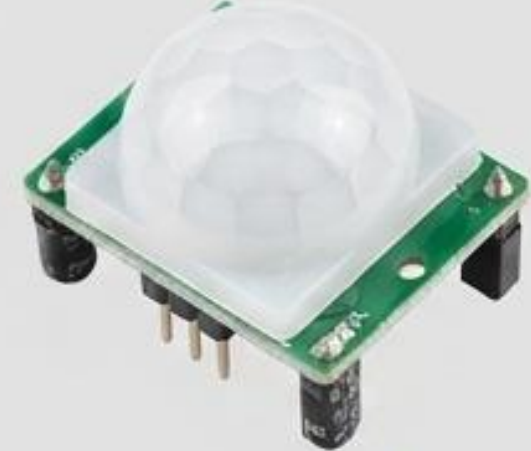
LDR (Light Sensor)

Role: Detects if it is bright or dark.



DHT22 (Temp/Humidity)

Role: Feels how hot or humid the room is.



PIR Motion Sensor

Role: "Peripheral Vision" that detects if a human walks by.

Outputs (The Muscles)

Outputs are how the microcontroller takes action and changes the physical world. Just like humans use muscles to move or a voice to speak, our Pico W uses Actuators.

**OUTPUTS = ELECTRICITY
FLOWING OUT OF THE PICO W**



The Action Hardware



LED

Role: The 'Voice' that turns on a light to alert us.



Servo Motor

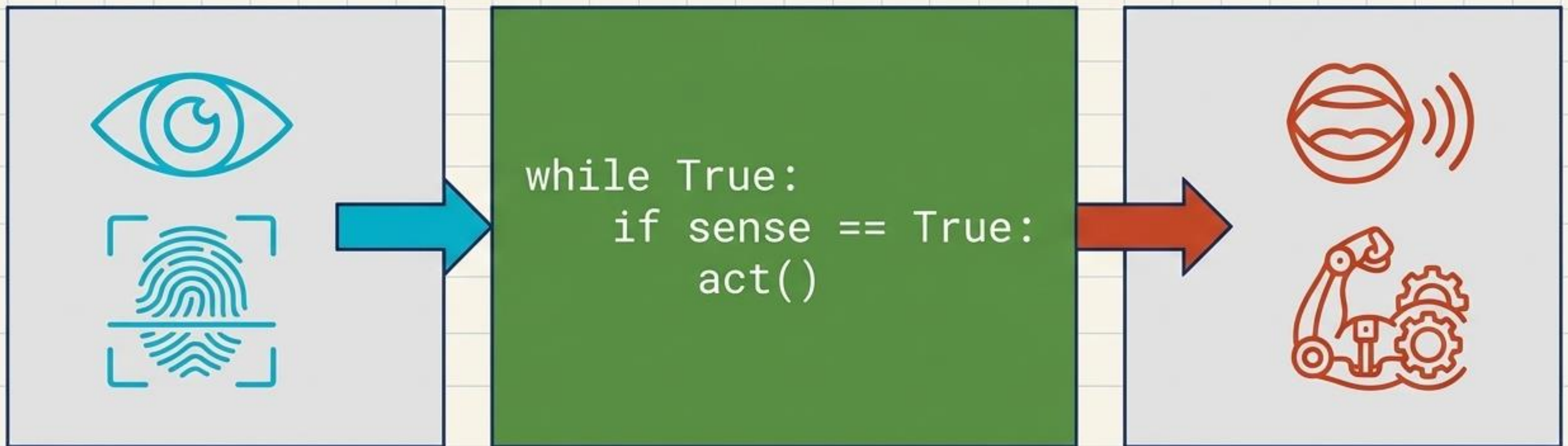
Role: The 'Muscles' that physically move a sunshade or curtain.

I/O Synthesis Architecture

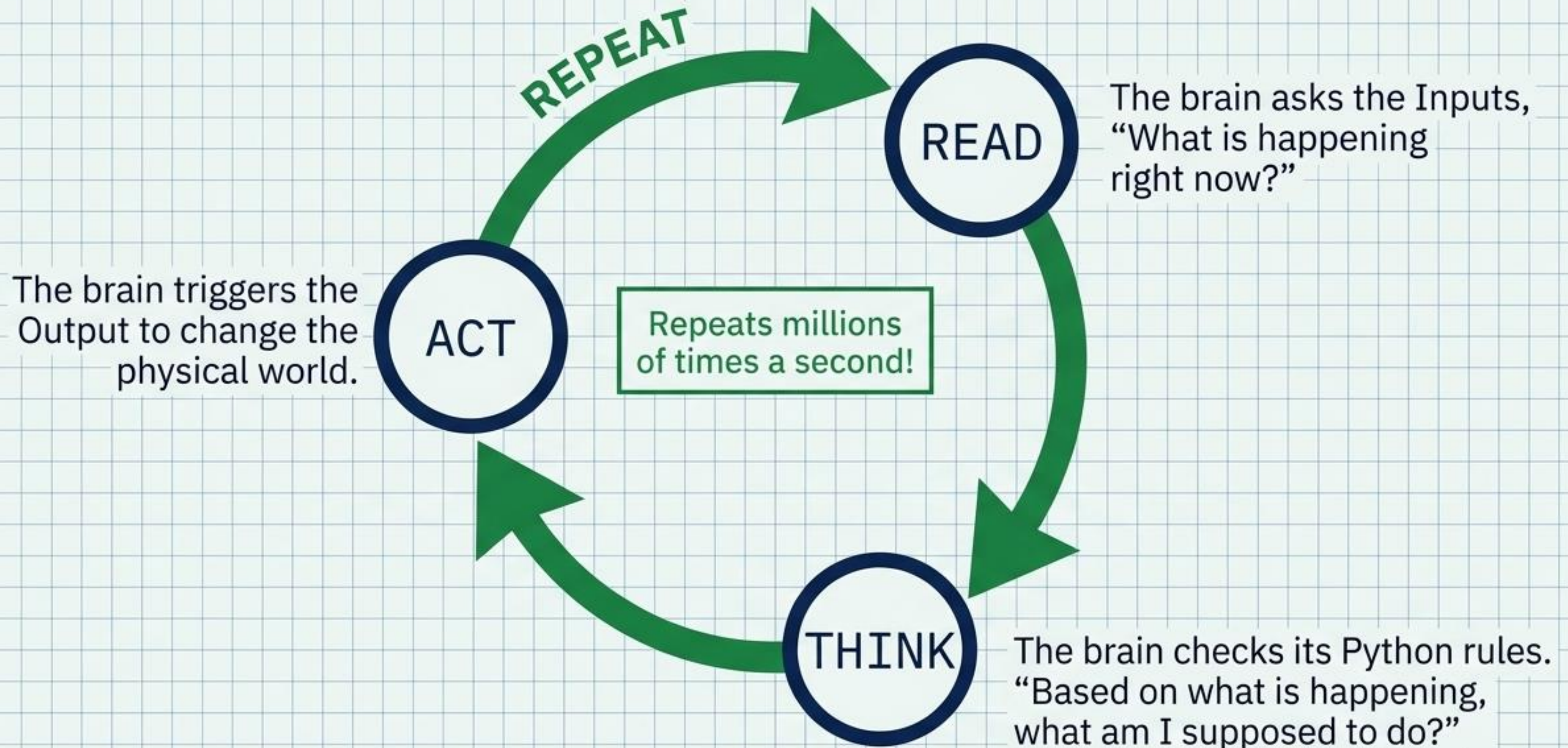
INPUTS	OUTPUTS
The Senses	The Muscles
Electricity flows INTO the Brain	Electricity flows OUT to the device
Sensors	Actuators
LDR, DHT22, PIR	LED, Servo Motor

The Software Link

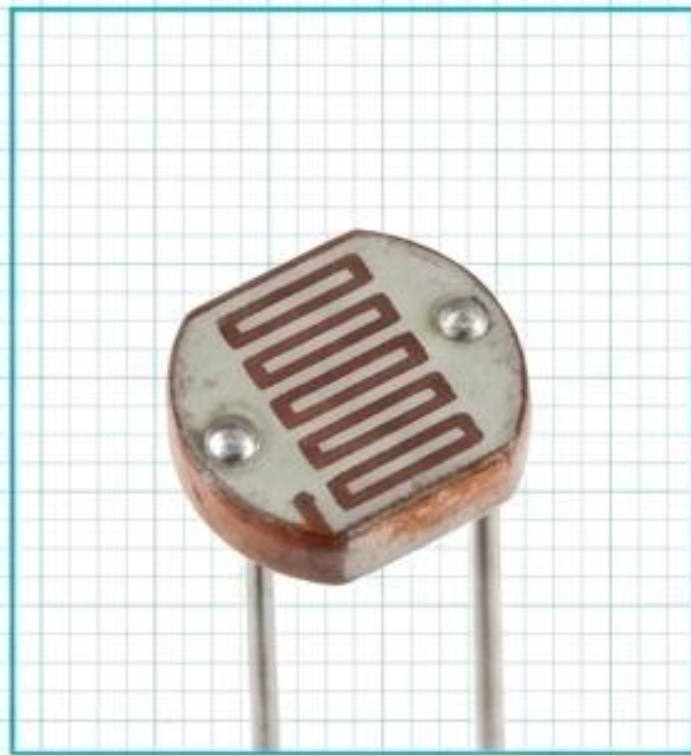
You have a Brain, Senses, and Muscles. But how does the system actually know what to do? It uses CircuitPython code to run an infinite loop, constantly connecting what it feels to what it does.



The Infinite Loop



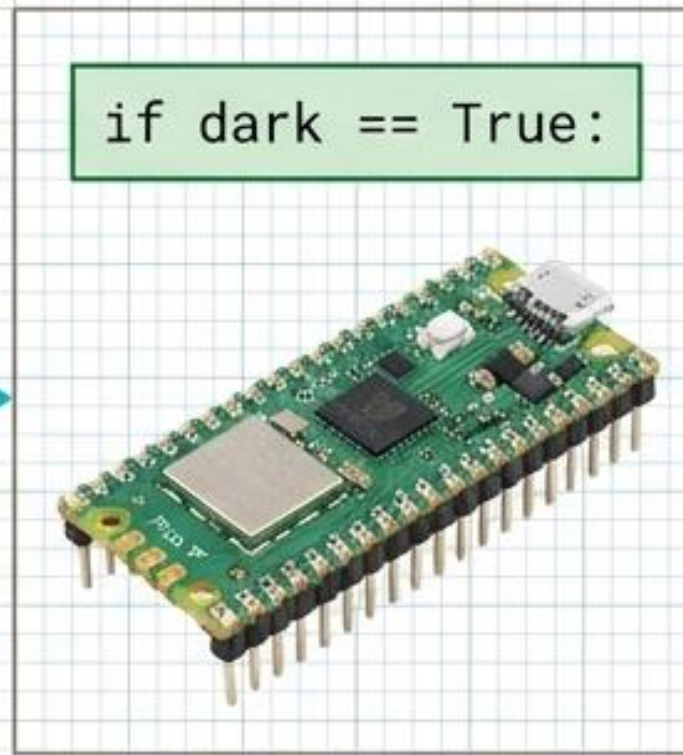
The Cyborg Blueprint in Action



It is getting dark outside.



(Electricity flows IN)



I need to alert the human.



Turn on the LED!
(Electricity flows OUT)



The loop immediately restarts to check if it is still dark.

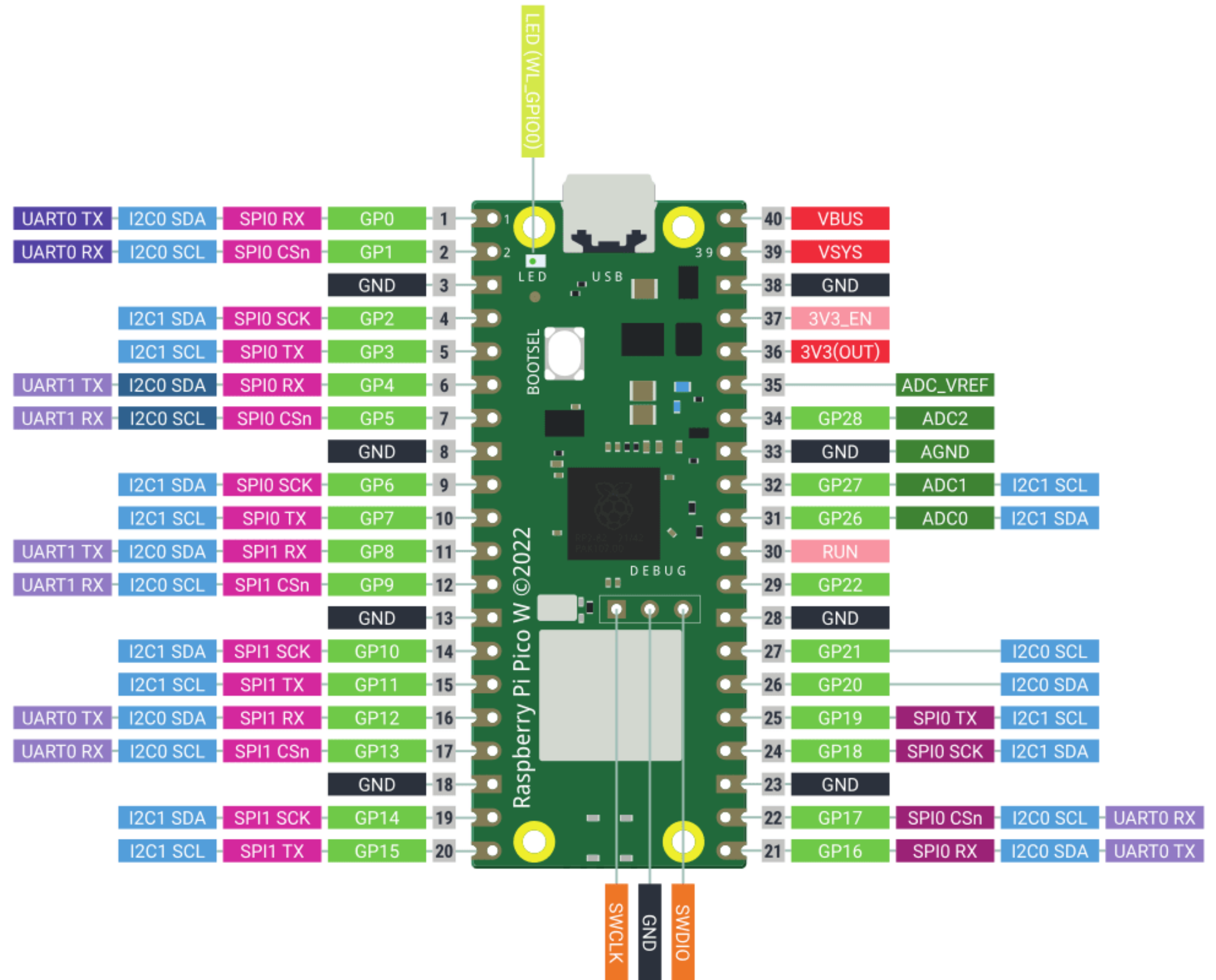
Pin Diagram

RP2040

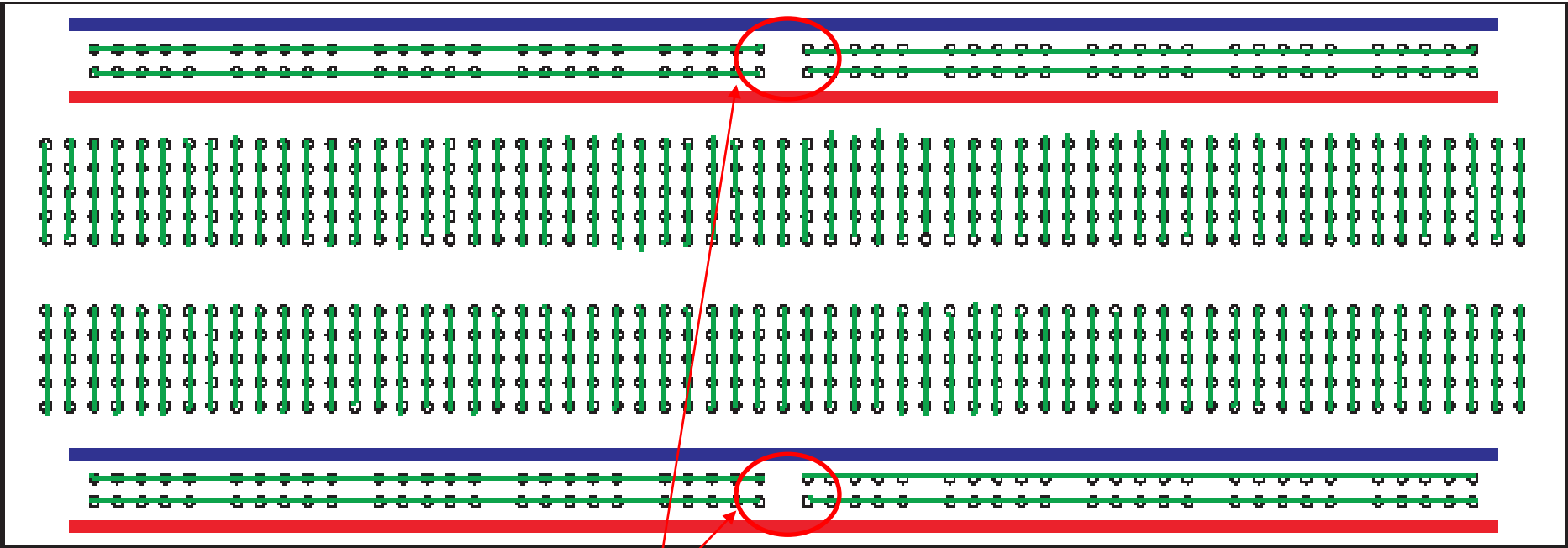
- Power
- Ground
- UART / UART (default)
- GPIO, PIO, and PWM
- ADC
- SPI / SPI (default)
- I2C / I2C (default)
- System Control
- Debugging

Infineon 43439

- GPIO



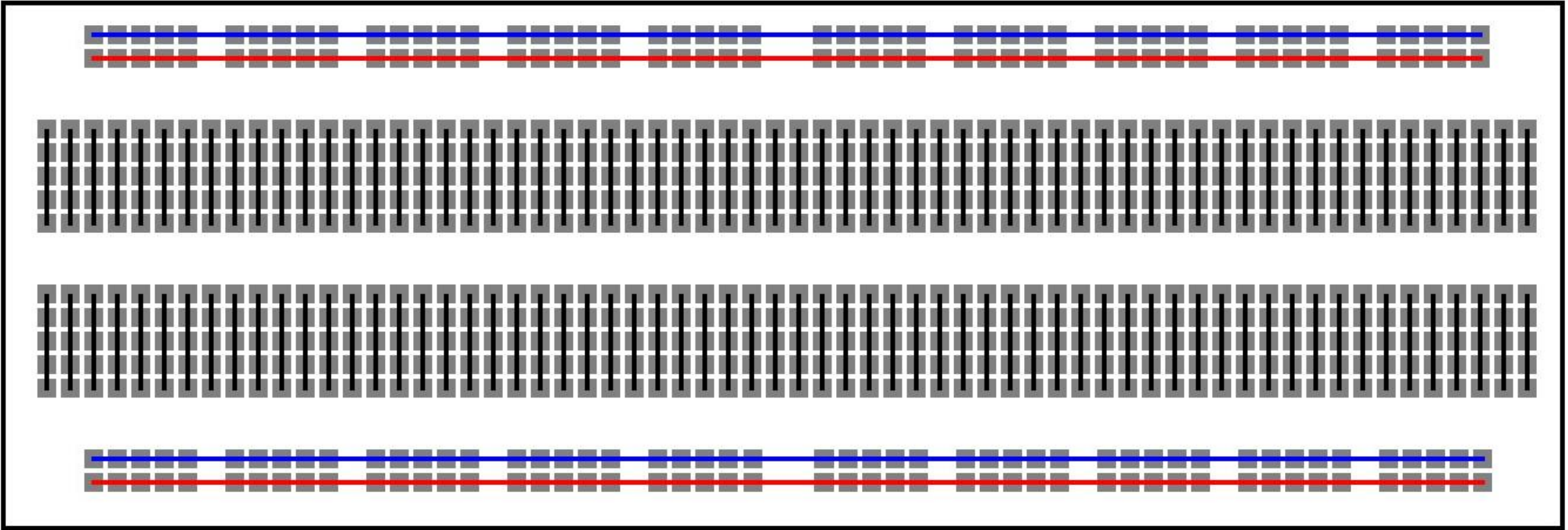
Breadboard

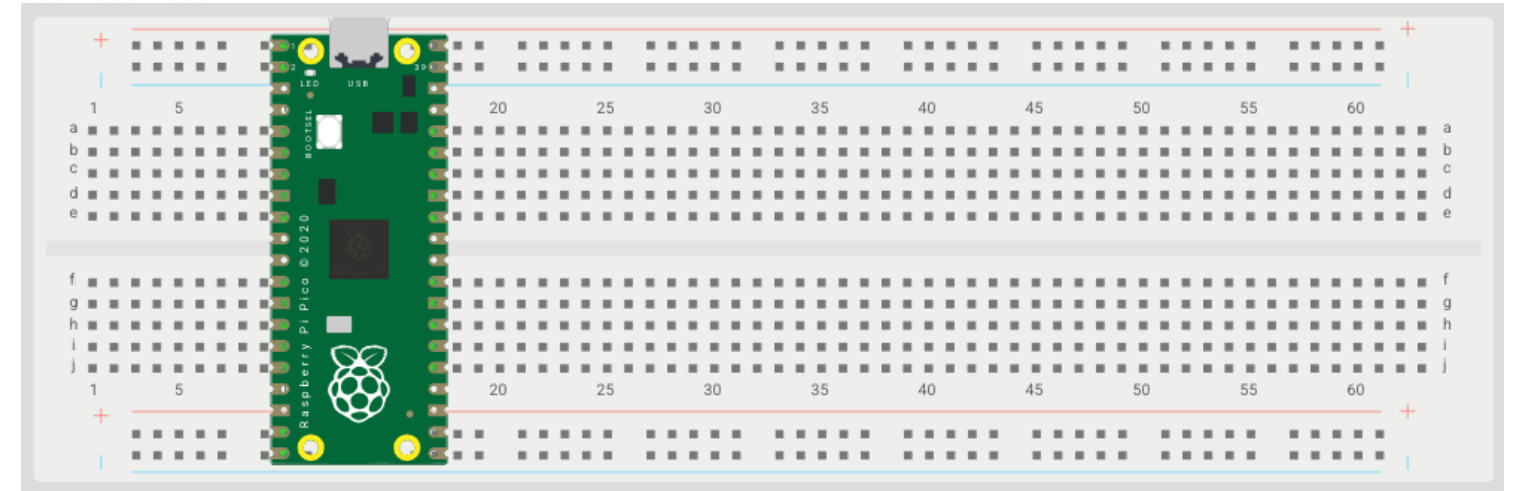
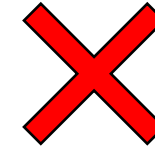


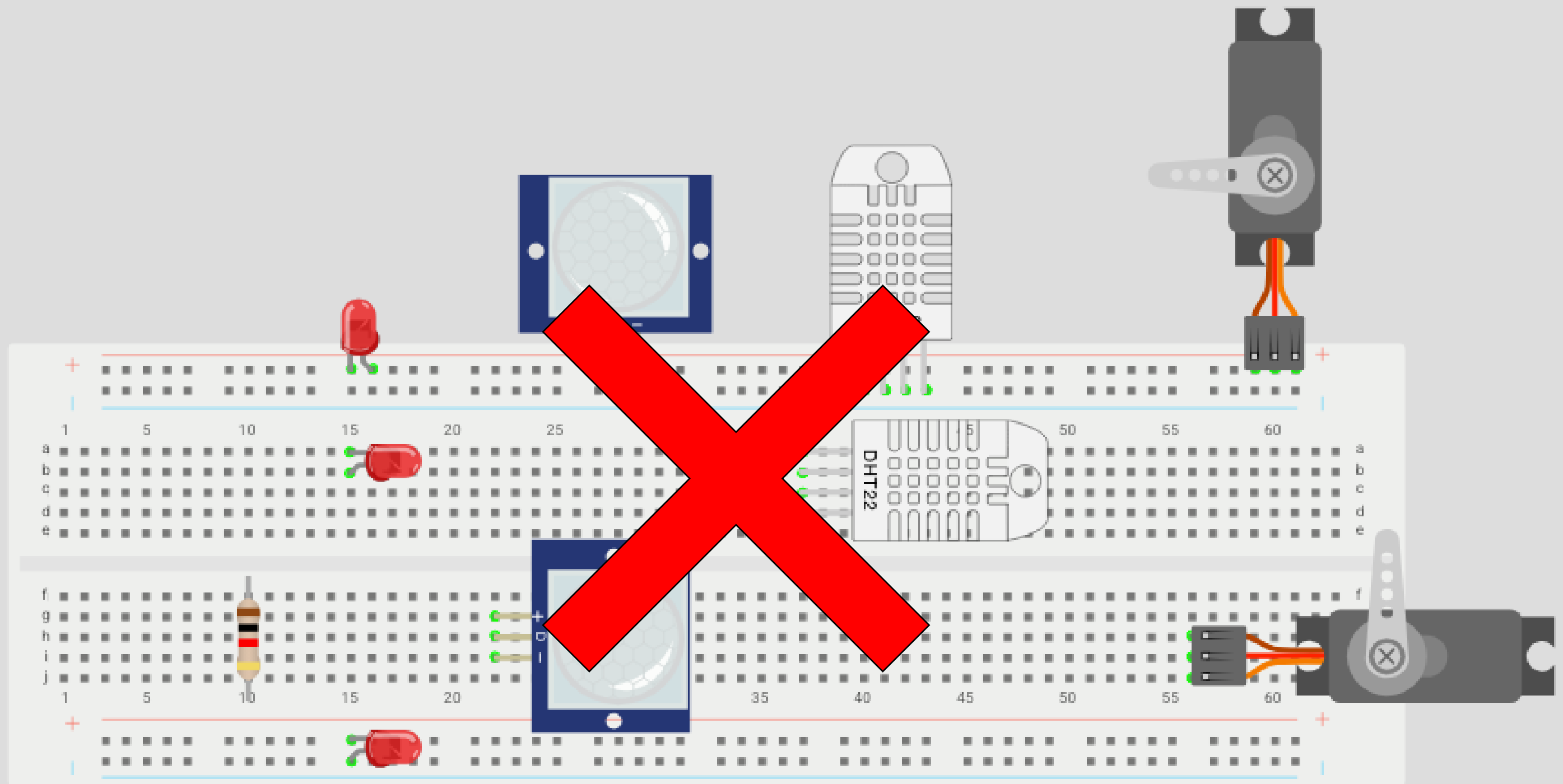
NO Connection

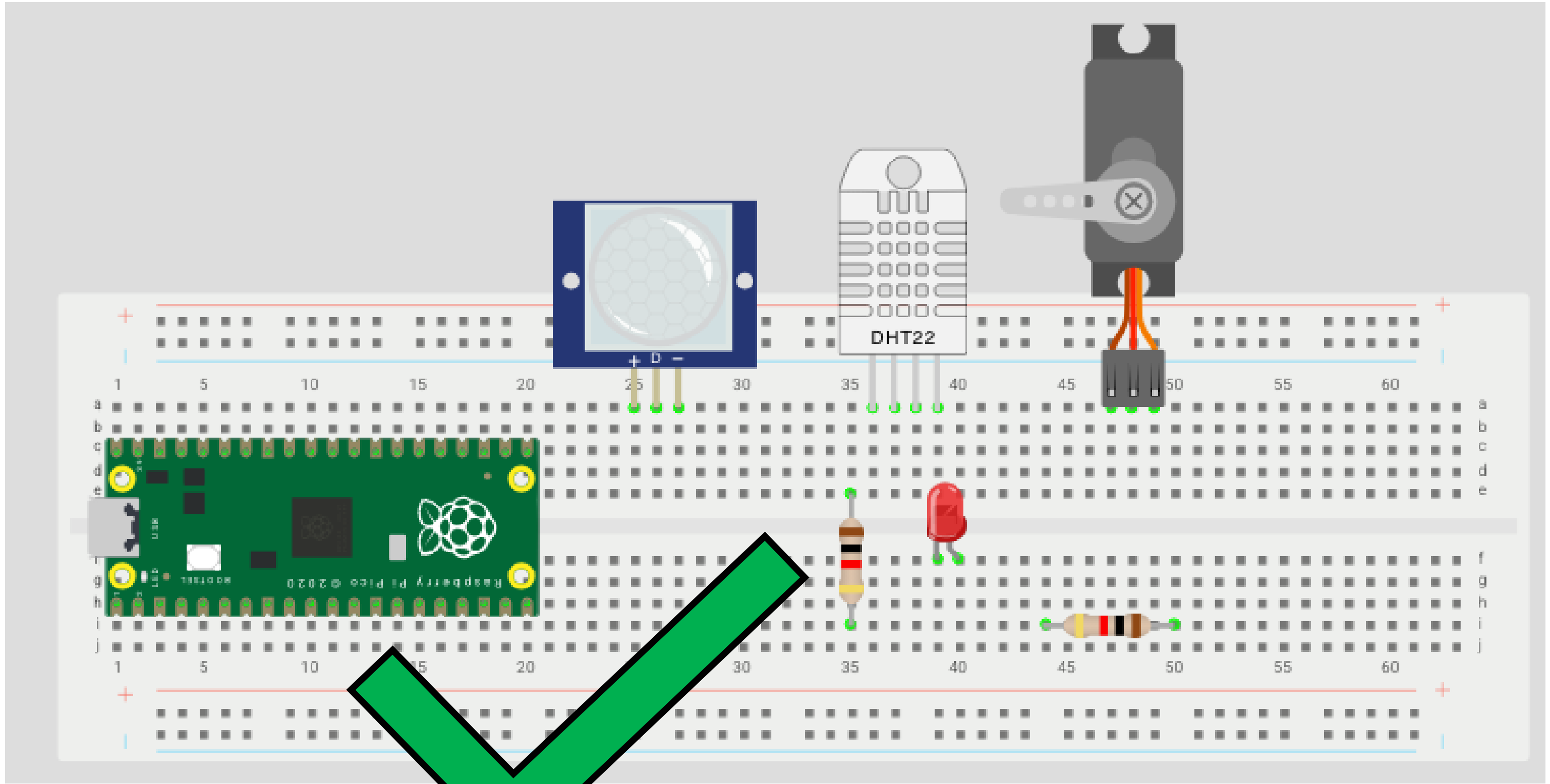
1

2





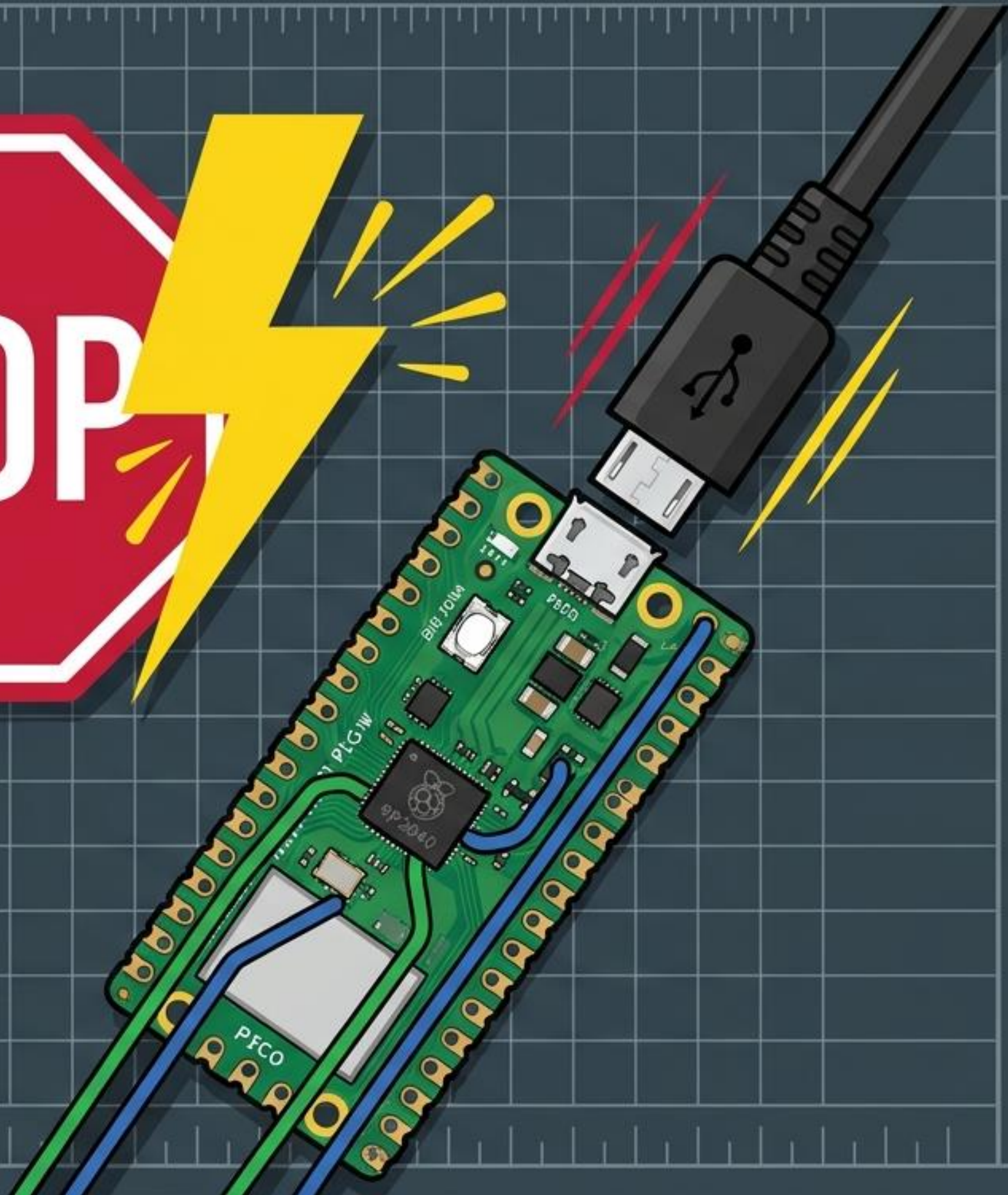




Wiring the IoT Hardware

A Hands-On Guide to Sensors, Actuators,
and CircuitPython with Raspberry Pi Pico W





The Golden Rule of Hardware Engineering

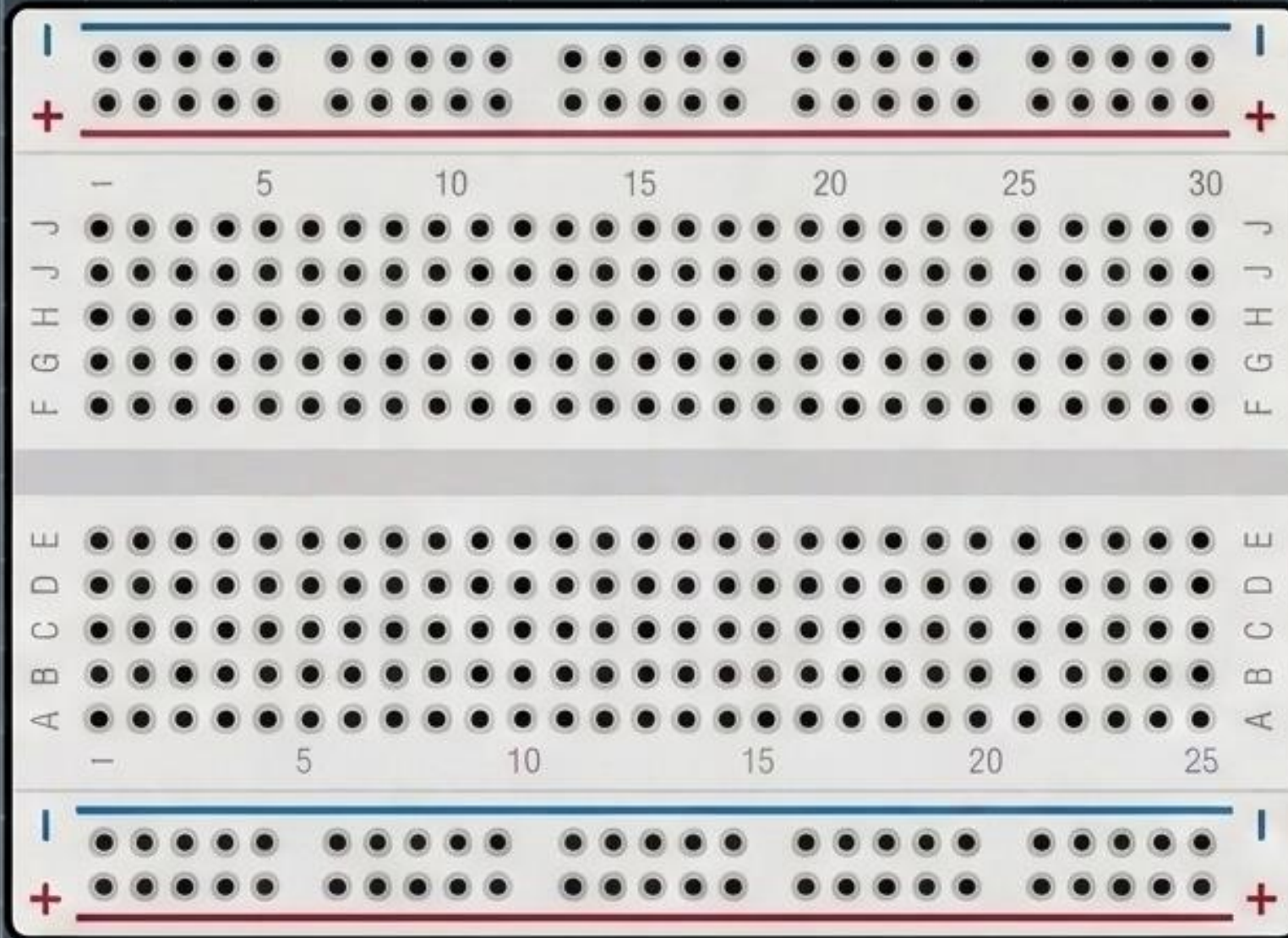
NEVER WIRE WITH THE POWER ON.

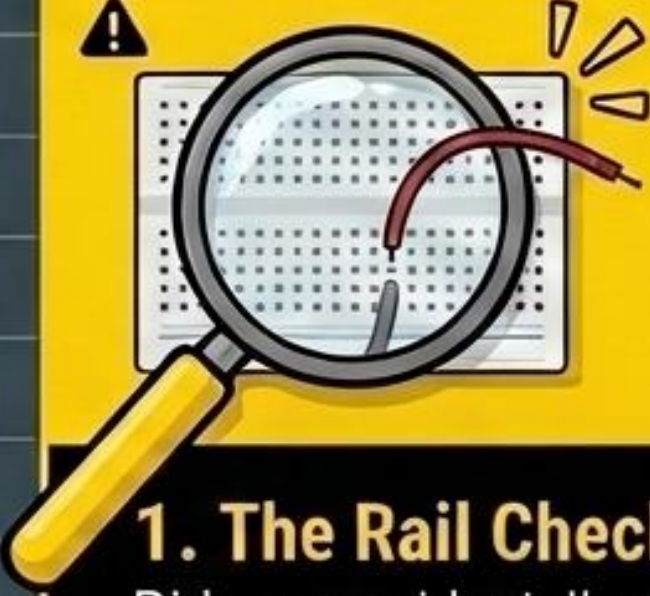
Always unplug the micro-USB cable from your Pico W before adding, removing, or moving jumper wires.

Accidental short circuits happen in a split second and can permanently fry your board.

Power off = Zero risk.

Master the Breadboard Before We Begin





Teacher Troubleshooting Guide: Universal Checks

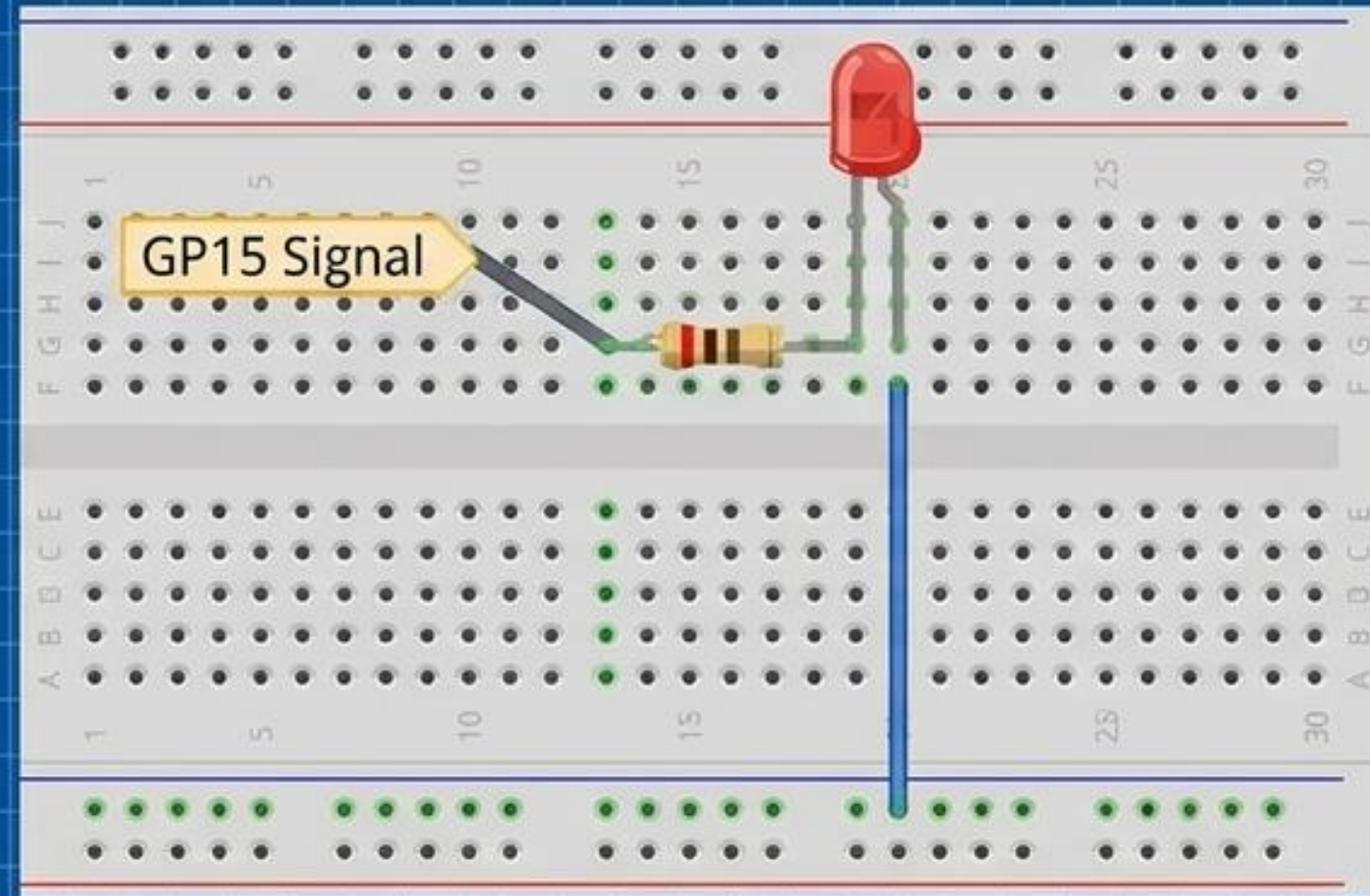
- 1. The Rail Check**

Did you accidentally swap your Power and Ground connections? Always double-check your red and black wires before powering up.
- 2. The Push Check**

Are all jumper wires pushed firmly into the holes? Loose wires are the #1 cause of broken code!

Component 1: The LED (Output Actuator)

Subtitle trmater for the and can diagram



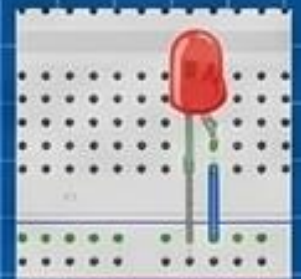
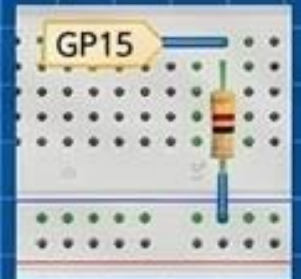
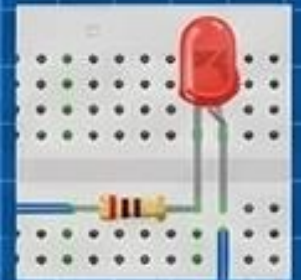
Wiring Steps

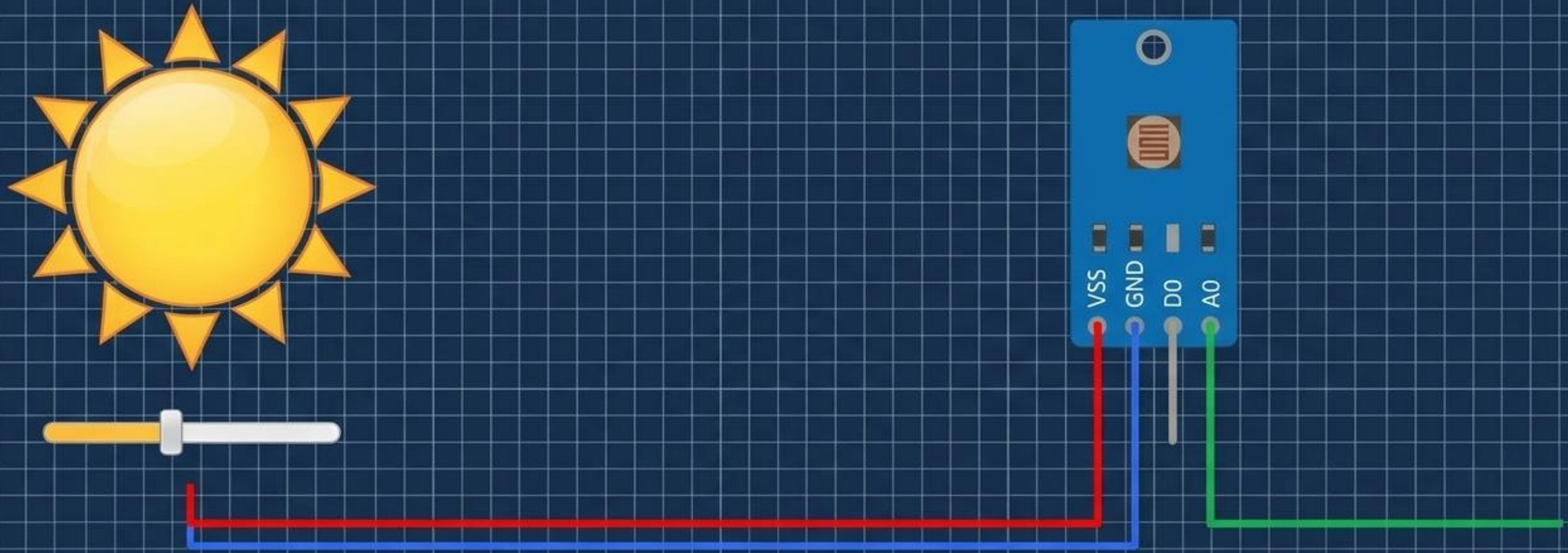
Step 1: Plug LED into breadboard

Step 2: Connect a 330Ω Resistor to the long leg (Anode).

Step 3: Connect the other end of the resistor to GP15.

Step 4: Connect the short leg (Cathode) to GND.

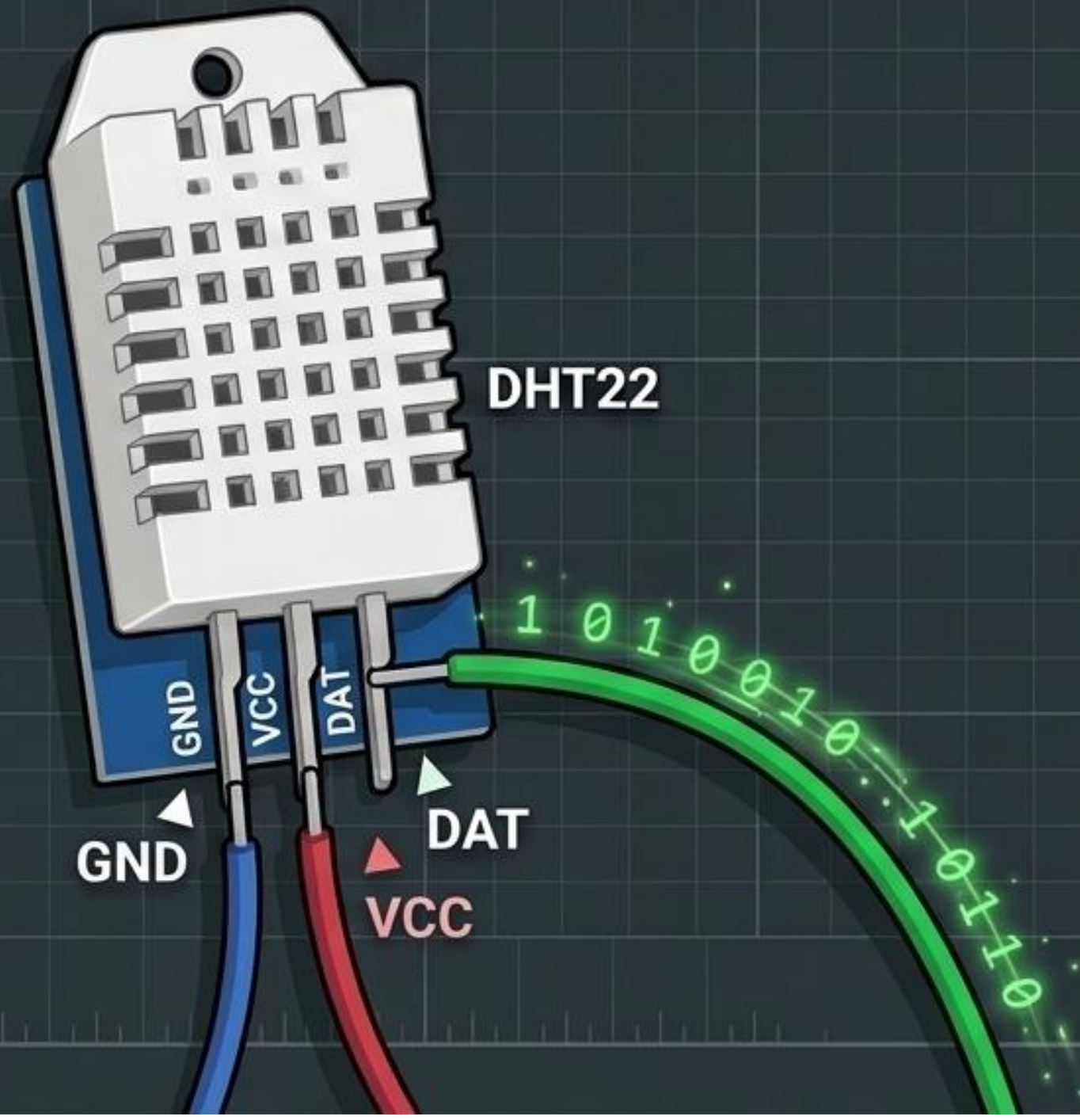




1. Connect VSS to the 3V3 pin (3.3 volts power).
2. Connect GND to any GND pin.
3. Connect A0 (Analog Output) to pin GP26 (Your ADC pin).

Component 3: DHT22 (Digital Input)

Measures air temperature and humidity, sending complex digital data packages down a single wire.



Wiring Steps:

1.

Connect **VCC (+)** to the **3V3** pin.

3V3

2.

Connect **GND (-)** to any **GND** pin.

GND

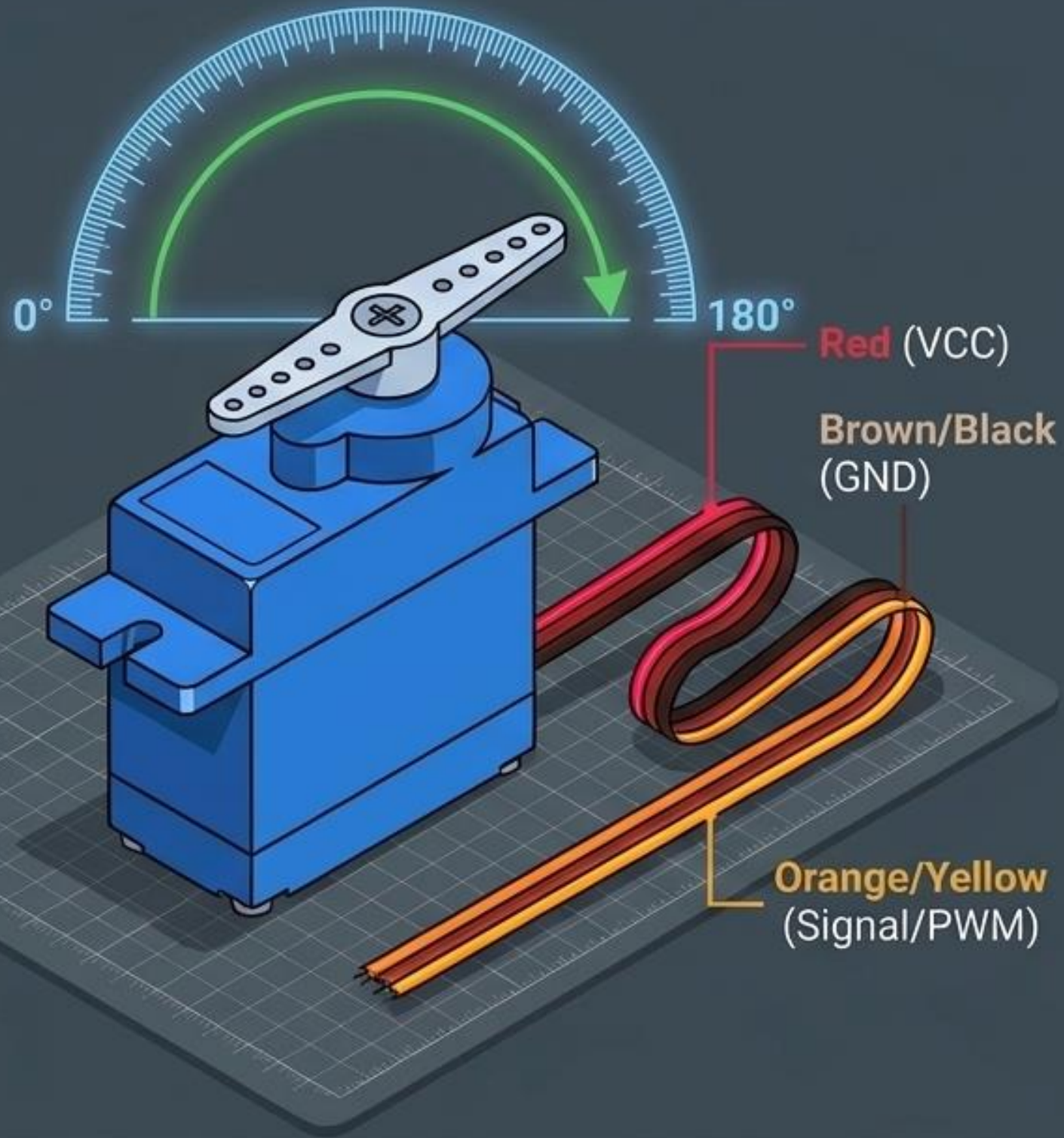
3.

Connect **DAT** to pin **GP14**.

GP14

Component 4: SG90 Micro Servo (Motor Actuator)

A motor programmed to turn to specific angles (0° to 180°) using PWM (Pulse Width Modulation).

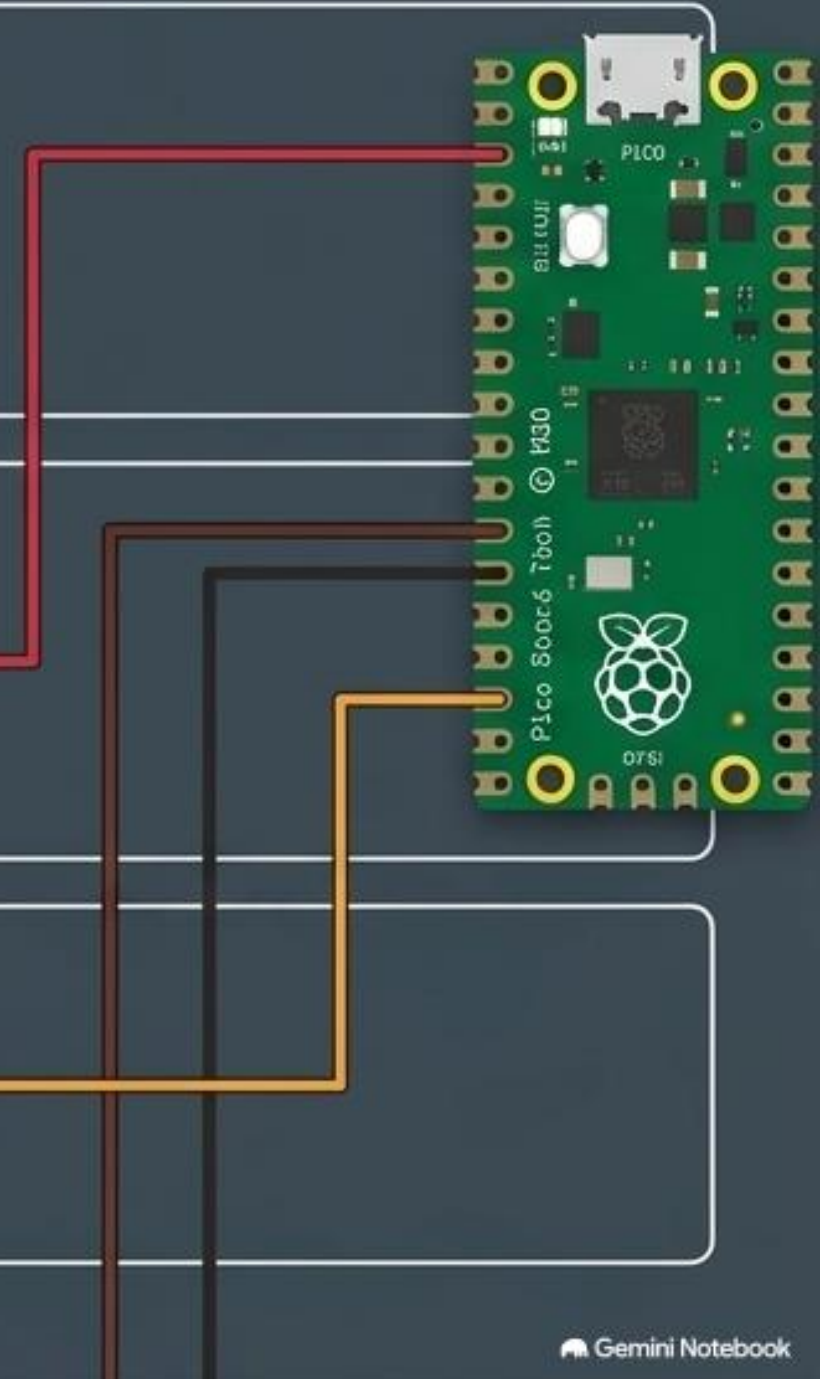


Wiring Steps:

1. Connect the **Red Wire** to the **3V3** pin.

2. Connect the **Brown/Black Wire** to GND.

3. Connect the **Orange/Yellow Wire** to pin GP16.



The Component Matrix

Component	Type	Power Needed	Signal Type	Key Pin
LED	Output Actuator	3.3V (w/ 220Ω Resistor)	Digital (On/Off)	GP15
LDR Module	Analog Input	3.3V	Analog (ADC)	GP26
DHT22 Module	Digital Input	3.3V	Digital (Data)	GP14
SG90 Servo	Motor Actuator	5.0V (VBUS) & 3.3V	PWM (Angles)	GP16

Ready to Code? The Final Pre-Flight Checklist

Before you plug that USB back in, verify these four critical checks:



The Golden Rule: Is the power still unplugged while you double-check your work?



The Rail Check: Are **Power** (Red) and **Ground** (Black) completely separated?



The Push Check: Are all jumper wires seated firmly into the breadboard?



The Code Check: Is your CircuitPython code saved as **code.py**?

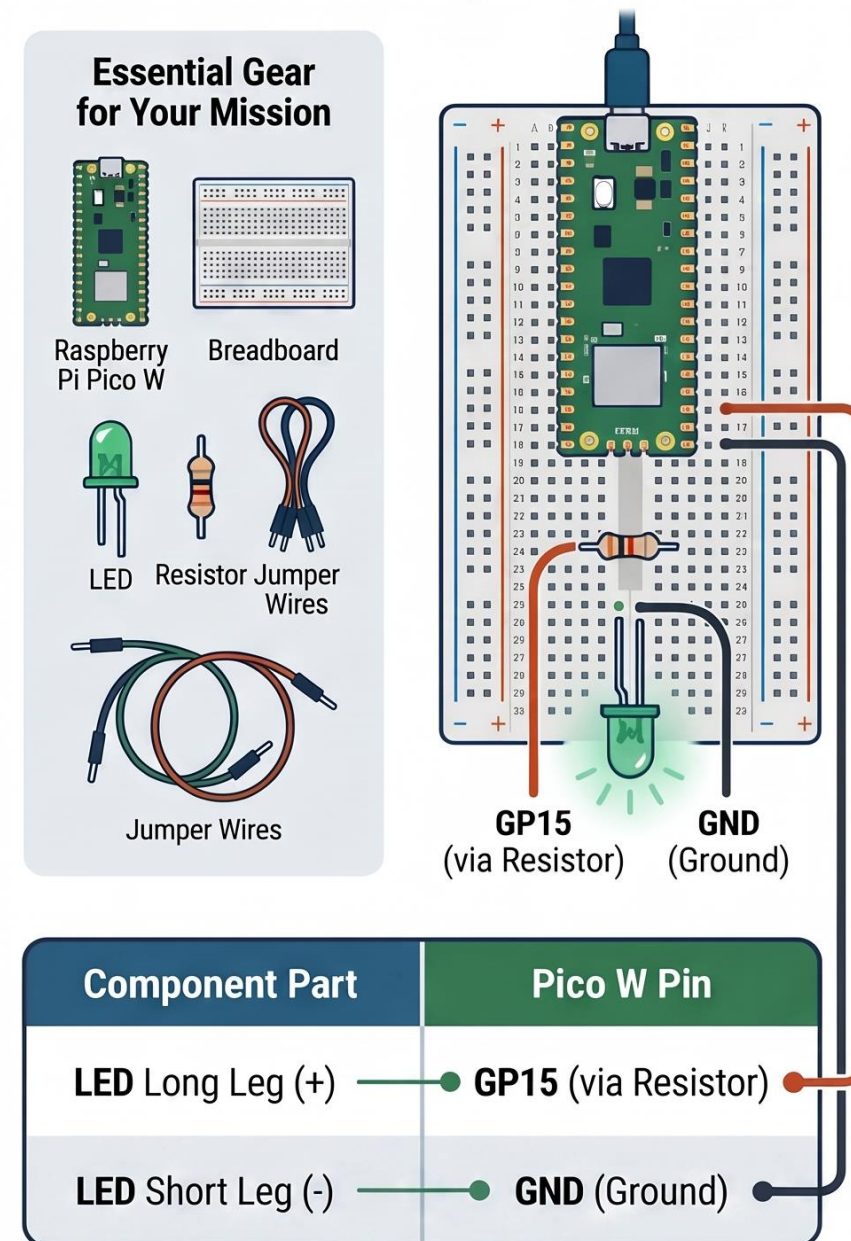
All checked? Plug in the USB and open CircuitPython!

Module 1

Hello Brain! Your First CircuitPython & GitHub Mission

Waking up the Raspberry Pi Pico W, running code, and documenting for GitHub.

Hardware & Wiring



Code & GitHub Deployment

Run code.py in Thonny

```
code.py
1 import board
2 import digitalio
3 import time
4
5 led = digitalio.DigitalInOut(board.GP15)
6 led.direction = digitalio.Direction.OUTPUT
7
8 while True:
9     led.value = True
10    time.sleep(0.5)
11    led.value = False
12    time.sleep(0.5)
13
```

Paste your code into Thonny and click **Run** to wake the hardware.

code.py

Upload Project to GitHub

Add your CircuitPython code and setup images to your GitHub repository.

setup-image.png

Project complete and shared!

Module 1

```
1 import board
2 import digitalio
3 import time
4
5 # 🔴 ===== 🔴
6 # 🔴          ENGINE ROOM: DO NOT TOUCH THE WIRES BELOW!          🔴
7 # 🔴 ===== 🔴
8
9 # 1. Plugging the smart light into the robot's brain
10 smart_light = digitalio.DigitalInOut(board.GP15)
11 smart_light.direction = digitalio.Direction.OUTPUT
12
13 print("🧠 Brain is awake! Starting the Smart Light...")
14
15
16 # 🎮 ===== 🎮
17 # 🎮          HACKER ZONE: SAFE TO EDIT!          🎮
18 # 🎮 ===== 🎮
19
20 # ⌚ HACKER MISSION: Change how fast the light blinks!
21 # Try changing these numbers to 0.5 (super fast!), 0.1 (strobe light!), or 3 (slow).
22 TIME_ON = 1
23 TIME_OFF = 1
24
25 while True:
26
27     # 1. Turn the light ON
28     smart_light.value = True
29     print("💡 Light is ON!")
30
31     # Wait while the light is on
32     time.sleep(TIME_ON)
33
34     # 2. Turn the light OFF
35     smart_light.value = False
36     print("🔴 Light is OFF!")
37
38     # Wait while the light is off
39     time.sleep(TIME_OFF)
```

Module 2

Sensing the World: Light-Triggered Motion with CircuitPython

Hardware & Wiring

Quick Reference: Pico W Connections

Component	Pico Pin	Wire Color/Type
LDR (Sensor)	GP26	Analog Input
Servo Signal	GP16	Orange Wire
Servo Power	VBUS (5V)	Red Wire

Software Mission

Library Setup:
Copy adafruit_motor to lib drive

File Setup:
Open code.py

Test & Calibrate:
Cover sensor or adjust thresholds

Adjust for sensitivity
(e.g., flashlight vs. ambient light)

CircuitPython & Adafruit Motor Library Compatible

Real-World Applications

Automated Curtains:
For smart home comfort

Automated Sunshades:
For smart agriculture

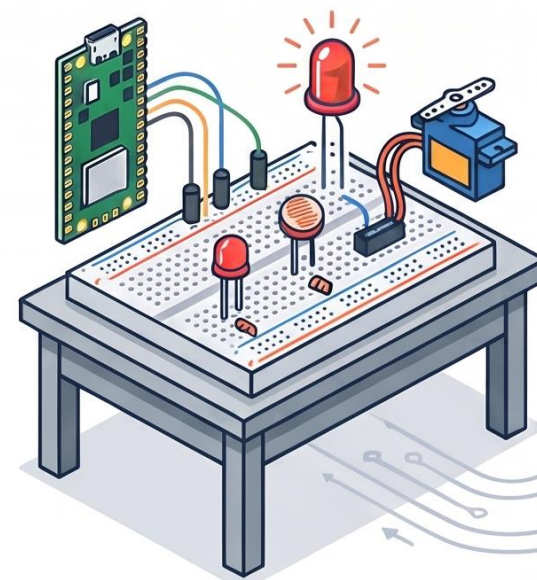
Module 2

```
1  import board, analogio, pwmio, time
2  from adafruit_motor import servo
3
4  # 🚫 ===== 🚫
5  # 🚫 ENGINE ROOM: DO NOT TOUCH THE WIRES BELOW! 🚫
6  # 🚫 ===== 🚫
7
8  # 1. Plugging in the robot's eye (Light Sensor)
9  ldr = analogio.AnalogIn(board.GP26)
10
11 # 2. Plugging in the robot's arm (Servo Motor)
12 pwm = pwmio.PWMOut(board.GP16, duty_cycle=0, frequency=50)
13 my_servo = servo.Servo(pwm)
14
15 # Robot memory: Is the curtain open or closed right now?
16 is_open = False
17
18 print("🤖 Robot Awake! Waiting for light...")
19
20
21 # 🎮 ===== 🎮
22 # 🎮 HACKER ZONE: SAFE TO EDIT! 🎮
23 # 🎮 ===== 🎮
24
25 # 🧑🔧 HACKER MISSION: Find your perfect "Magic Number"!
26 # Use your flashlight and your hand to test the light sensor.
27 # Change 50 to the best percentage for your room (0 to 100).
28 MAGIC_NUMBER = 50
29
```

```
30 while True:
31
32     # 1. Read the light sensor (Flipping the numbers and converting to percentage 0-100%)
33     inverted_value = 65535 - ldr.value
34     light_level = (inverted_value / 65535) * 100
35
36     # We use :.1f to round it to 1 decimal place (e.g., 45.2%)
37     print(f"☀️ Current Light Level is: {light_level:.1f}%")
38
39     # 2. The Robot's Brain (If / Else Decision)
40     if light_level > MAGIC_NUMBER and is_open == False:
41         # BRIGHT LIGHT DETECTED!
42         print("😎 It is bright! Opening the curtains...")
43         my_servo.angle = 90 # 90 degrees is OPEN
44         is_open = True     # The robot remembers it opened the curtain
45
46     elif light_level <= MAGIC_NUMBER and is_open == True:
47         # DARKNESS DETECTED!
48         print("😓 It is dark! Closing the curtains...")
49         my_servo.angle = 0  # 0 degrees is CLOSED
50         is_open = False    # The robot remembers it closed the curtain
51
52     # Wait 1 second before checking again so the robot doesn't get dizzy
53     time.sleep(1)
```

Module 3

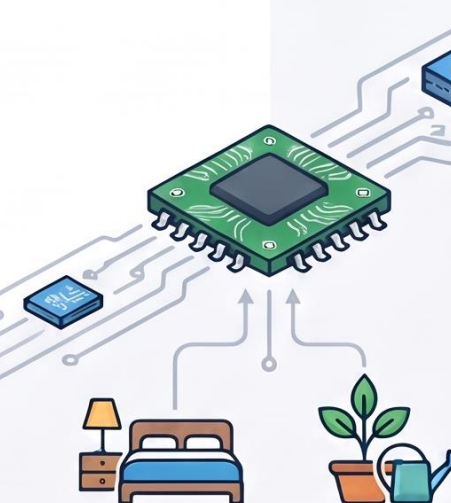
Pico IoT Adventure: From Hardware to Cloud



Phase 1: Local Control (Modules 1-2)

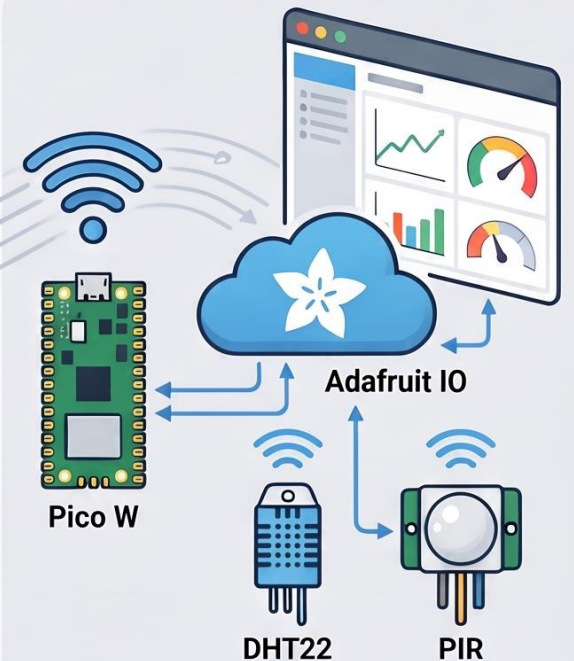
Master GPIO outputs and analog inputs using LEDs, LDRs, and Servo Motors.

Focus Hardware: LED, LDR, Servo.
Key Concept: GPIO, Logic, and Inputs.



Real-World Application Themes

Students apply skills to either a Smart Bedroom or a Mini Smart Farm.



Phase 2: Cloud Connectivity (Modules 3-4)

Link sensors to WiFi and build browser-based dashboards for remote monitoring.

Focus Hardware: DHT22, PIR, WiFi.
Key Concept: MQTT, WiFi, and HTML/JS.



The Brain: Raspberry Pi Pico W

Uses CircuitPython for easy-to-learn, beginner-friendly coding in Thonny IDE.



Smart Sensing Trio

Includes LDR (Light), DHT22 (Climate), and PIR (Motion) sensors.



Cloud Power: Adafruit IO

Provides a free MQTT broker and dashboard for visual data storage.

Module 3



<https://github.com/mymadi/pico-iot-adventure>

Module 4

Module 4: Launching Your Smart Control Center

The Mission: Setup & Control

1. Link Your Live Data

```
<html>
<meta charset="UTF-8">
const username = "Your_Adafruit_IO_Username";

const username = "Your_Adafruit_IO_Username";

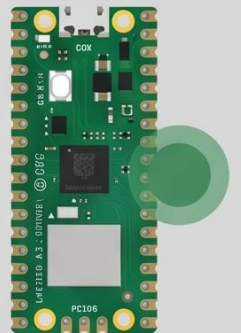
<div class="wrapper">
<div class="button">
<button>Light</button>
</div>
<div class="Text out">
</div>
</div>
</html>
```

Open index.html and enter your Adafruit IO username to sync with your hardware.

2. Test Physical Control

TURN ON LIGHT

Click the "Turn On Light" button to trigger the physical LED on your Pico W.



- Hardware Requirements**
- ✓ Ensure your Pico W is powered on
 - ✓ Connected to the WiFi network.

The Hacker Challenge: Customization

1. Edit the Source Code

<!-- Open index.html with a text editor -->

Open the index.html file using a basic text editor like Notepad.

2. Personalize Your Dashboard

My Dashboard

My Awesome Smart Bedroom

Locate the title tag and rename it to "My Awesome Smart Bedroom."

Module 4

 **IoT Dashboard**

Adafruit Username

Adafruit AIO Key

Connect

Temperature
-- °C

Humidity
-- %

Light Level
--

Motion Status
--

Remote Control

Turn LED ON

Module 5



AI Smart Dashboard

Connect

LATEST AI DECISION

"Waiting for system connection..."



TEMPERATURE

-- °C



HUMIDITY

-- %



LIGHT LEVEL

--



MOTION STATUS

--



AI / MANUAL OVERRIDE

Turn LED ON

Module 5

Building the Smart AI Brain: Autonomous Edge AI with CircuitPython

Transform a Raspberry Pi Pico W into an independent 'Expert System' with real-time sensor data processing.

HARDWARE SETUP & WIRING: SENSORY INPUTS

The Sensor Suite

-  **Temp/Humidity:** ``DHT22`` (3V3)
-  **Motion:** ``PIR`` (5V)
-  **Light:** ``LDR`` (Analog, ``GP26``)

Master Wiring Reference for Pico W GPIO Pins

	Component	Pico W Pin	Notes
1	LED	<code>`GP15`</code>	Use 220Ω Resistor
2	Servo Motor	<code>`GP16`</code>	Orange to <code>`GP16`</code> ; Red to 5V
3	LDR Sensor	<code>`GP26`</code>	Connect to Analog Pin

Output Configuration

LED Response: ``GP15``

Physical Action: ``SG90 Servo`` (``GP16``)

Powering Components

Ensure Servo/PIR use ``VBUS`` (5V); sensors like ``DHT22`` use 3V3.

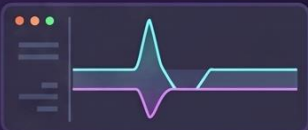
THE AI LOGIC ENGINE: AUTONOMOUS DECISION MAKING

The Expert System

Edge AI using Boolean logic to process sensor data and make instant decisions locally.

Calibration Thresholds

Define specific triggers:
`DARK_THRESHOLD = 30%` (LDR)



or
`HOT_THRESHOLD = 28°C` (DHT22) to "train" behavior.



Autonomous Reporting

Broadcast all "thoughts" and actions to the **Adafruit IO** `'ai-decision'` feed for real-time remote visualization.



Thank You

